3.3: Digital Input and Output (I/O)

Introduction to Digital I/O

Digital Input/Output (I/O) is a fundamental concept in microcontroller programming, allowing your Arduino to interact with the physical world. Digital signals are binary, meaning they can only be in one of two states: HIGH or LOW. These states typically correspond to a voltage level, such as 5V (HIGH) and 0V (LOW) for many Arduino boards.

In this module, we will explore the core functions used for digital I/O on Arduino: `pinMode()`, `digitalWrite()`, and `digitalRead()`. Understanding these functions is crucial for controlling external components like LEDs and reading input from sensors or buttons.

3.3.1: `pinMode()` Function

The `pinMode()` function is used to configure a specific pin to behave either as an input or an output. This is one of the first functions you'll typically use in your `setup()` routine for any pin you intend to use for I/O.

Syntax

```
pinMode(pin, mode);
```

Parameters

- `pin`: The number of the digital pin whose mode you wish to set. (e.g., `2`, `13`, `A0` for analog pins used as digital).
- `mode`: The mode to set the pin to. This can be one of three values:

- `INPUT`: Configures the specified pin to receive incoming signals. When a pin is configured as `INPUT`, it has a high impedance, meaning it draws very little current.
- `OUTPUT`: Configures the specified pin to send outgoing signals. When a pin is configured as `OUTPUT`, it is in a low-impedance state and can source or sink a significant amount of current.
- `INPUT_PULLUP`: Configures the specified pin as an input with an internal pull-up resistor enabled. This is useful for connecting buttons or switches without needing an external resistor.

Example

To set digital pin 13 as an output (for an LED) and digital pin 2 as an input with an internal pull-up resistor (for a button):

```
void setup() {
  pinMode(13, OUTPUT);      // Set pin 13 as an output
  pinMode(2, INPUT_PULLUP); // Set pin 2 as an input with pull-up resistor
}

void loop() {
  // main program here
}
```

Internal Pull-up Resistors

When a pin is configured as `INPUT_PULLUP`, an internal resistor is connected between the pin and the VCC (5V) line. This ensures that the input reads HIGH when nothing is connected or when a switch is open. When a switch connects the pin to ground, the input reads LOW.

Mode	Description	Typical Use Case
`INPUT`	High impedance, reads external voltage.	Reading sensors, external signals.
`OUTPUT`	Low impedance, provides voltage (HIGH) or ground (LOW).	Controlling LEDs, motors, relays.
`INPUT_PULLUP`	High impedance with internal pull-up resistor, reads external voltage.	Reading buttons, switches (connected to ground).

3.3.2: `digitalWrite()` Function (Controlling LEDs)

The `digitalWrite()` function is used to write a HIGH or LOW value to a digital pin that has been configured as an `OUTPUT` with `pinMode()`.

Syntax

```
digitalWrite(pin, value);
```

Parameters

- `pin`: The number of the digital pin you wish to write to.
- `value`: Either `HIGH` or `LOW`.
  - `HIGH`: Sets the pin to the voltage level corresponding to 1 (e.g., 5V).
  - `LOW`: Sets the pin to the voltage level corresponding to 0 (e.g., 0V or ground).

Example: Blinking an LED

This classic Arduino example demonstrates how to blink an LED connected to digital pin 13.

```
void setup() {
  pinMode(13, OUTPUT); // Initialize digital pin 13 as an output.
}

void loop() {
  digitalWrite(13, HIGH); // Turn the LED on (HIGH is the voltage level)
  delay(1000);            // Wait for a second
  digitalWrite(13, LOW);  // Turn the LED off by making the voltage LOW
  delay(1000);            // Wait for a second
}
```

Explanation

In the `setup()` function, `pinMode(13, OUTPUT)` configures pin 13 to send signals. In the `loop()` function, `digitalWrite(13, HIGH)` turns the LED on, `delay(1000)` pauses the program for 1000 milliseconds (1 second), `digitalWrite(13, LOW)` turns the LED off, and another `delay(1000)` pauses for another second. This creates a blinking effect.

3.3.3: `digitalRead()` Function (Reading Buttons, Debouncing)

The `digitalRead()` function is used to read the value from a specified digital pin. The value returned will be either `HIGH` or `LOW`.

Syntax

```
digitalRead(pin);
```

Parameters

- `pin`: The number of the digital pin you wish to read from.

Returns

- `HIGH` : If the pin is high (e.g., 5V).
- `LOW` : If the pin is low (e.g., 0V).

Example: Reading a Button

This example shows how to read the state of a button connected to digital pin 2 and turn on an LED on pin 13 when the button is pressed.

```
const int buttonPin = 2;  // the number of the pushbutton pin
const int ledPin = 13;    // the number of the LED pin

void setup() {
  pinMode(ledPin, OUTPUT);      // initialize the LED pin as an output:
  pinMode(buttonPin, INPUT_PULLUP); // initialize the pushbutton pin as an
input with pull-up:
}

void loop() {
  // read the state of the pushbutton value:
  int buttonState = digitalRead(buttonPin);

  // check if the pushbutton is pressed. If it is, the buttonState is LOW:
  if (buttonState == LOW) {
    // turn LED on:
    digitalWrite(ledPin, HIGH);
  } else {
    // turn LED off:
    digitalWrite(ledPin, LOW);
  }
}
```

Debouncing

When a mechanical button is pressed or released, the contacts can bounce, causing rapid, unwanted HIGH/LOW transitions before settling into a stable state. This phenomenon is called **switch bounce** or **chatter**. If not handled, the microcontroller might register multiple presses for a single physical press.

Debouncing is the process of eliminating these spurious readings. A common software debouncing technique involves waiting a short period (e.g., 50 milliseconds) after detecting a change in state to ensure the signal has stabilized.

Software Debouncing Example

```
const int buttonPin = 2;     // the number of the pushbutton pin
const int ledPin = 13;       // the number of the LED pin

int buttonState;             // current state of the button
int lastButtonState = HIGH;  // previous state of the button

long lastDebounceTime = 0;   // the last time the output pin was toggled
long debounceDelay = 50;     // the debounce time; increase if the output
flickers

void setup() {
  pinMode(buttonPin, INPUT_PULLUP);
  pinMode(ledPin, OUTPUT);
  Serial.begin(9600); // Initialize serial communication for debugging
}

void loop() {
  int reading = digitalRead(buttonPin);

  // If the switch changed, due to noise or pressing:
  if (reading != lastButtonState) {
    // reset the debouncing timer
    lastDebounceTime = millis();
  }

  if ((millis() - lastDebounceTime) > debounceDelay) {
    // whatever the reading is at, it's been there for longer than the
debounce
    // delay, so take it as the actual current state:

    // if the button state has changed:
    if (reading != buttonState) {
      buttonState = reading;

      // only toggle the LED if the new button state is LOW
      if (buttonState == LOW) {
        digitalWrite(ledPin, !digitalRead(ledPin)); // Toggle LED
        Serial.println("Button Pressed!");
      }
    }
  }
```

```
  lastButtonState = reading;
}
```